

Mediator - Lớp trung gian

Mẫu thiết kế - SE401.Q21

23521224 Trương Hoàng Phúc

23521140 Lê Minh Phát

22520243 Ya Đạt

1	Tên & Phân loại	4
2	Mục đích & Ý định	6
3	Vấn đề	8
4	Giải pháp	10
5	Cấu trúc	12
6	Luồng hoạt động	15
7	Ví dụ thực tế	17
8	Mã nguồn minh họa	19
9	Ứng dụng thực tế	23
10	Trường hợp áp dụng	25
11	Ưu nhược điểm	27
12	Cách áp dụng	29
13	So sánh với các mẫu khác	31

14	Các mẫu liên quan	33
15	Lỗi thường gặp	35
16	Tóm tắt	37

1 Tên & Phân loại

- Tên gọi: Mediator, lớp trung gian
- Tên khác: Intermediary, Controller
- Phân loại: Behavioral Pattern (Mẫu ứng xử)

Các mẫu giải quyết vấn đề liên quan đến giao tiếp và trách nhiệm giữa các đối tượng

2 Mục đích & Ý định

Mục đích chính: Giảm sự phụ thuộc lẫn nhau giữa các lớp, đảm nhiệm việc làm lớp trung gian thực hiện việc giao tiếp gián tiếp giữa các lớp.

Ý định:

- Giải phóng các đối tượng khỏi sự phụ thuộc chặt chẽ
- Tạo điểm tập trung duy nhất cho logic giao tiếp
- Tăng khả năng tái sử dụng của các đối tượng

3 Vấn đề

Tình huống: Dialog chỉnh sửa hồ sơ khách hàng

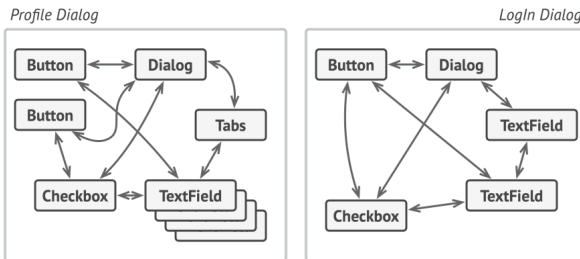


Figure 1: Các phần tử UI có quan hệ phức tạp với nhau

Vấn đề: Mỗi phần tử phải liên lạc trực tiếp với nhiều phần tử khác

4 Giải pháp

Ý tưởng: Dùng giao tiếp trực tiếp, sử dụng Mediator

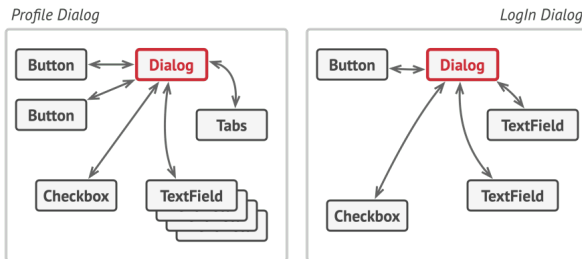


Figure 2: Giao tiếp gián tiếp qua Mediator

Lợi ích:

- Các thành phần chỉ phụ thuộc vào 1 Mediator
- Giảm phụ thuộc lẫn nhau, tập trung logic giao tiếp

5 Cấu trúc

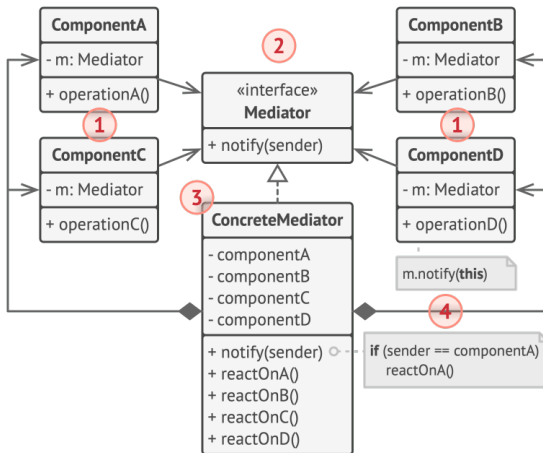


Figure 3: Biểu đồ cấu trúc của mẫu thiết kế Mediator

1. **Các lớp Component:** chứa business logic, sử dụng Mediator qua interface. Không biết lớp Mediator cụ thể, dễ tái sử dụng.
2. **Mediator trừu tượng:** Khai báo phương thức giao tiếp, thường chỉ có `notify(sender)`.
3. **Mediator kế thừa:** quản lý quan hệ giữa các Component, tham chiếu đến các lớp Component, thực hiện logic giao tiếp phức tạp.
4. Component không biết nhau, chỉ thông báo đến Mediator khi có sự kiện. Mediator xử lý và gọi Component khác.

6 Luồng hoạt động

1. Component gọi `Mediator.notify(this)`
2. Mediator nhận thông báo từ Component
3. Mediator xác định Component cần nhận thông báo
4. Mediator gọi phương thức thích hợp trên Component khác
5. Các Component không biết nhau, chỉ biết Mediator

7 Ví dụ thực tế

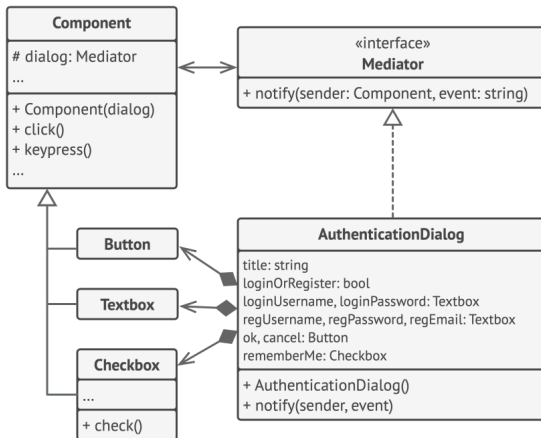


Figure 4: Áp dụng mẫu Mediator vào hộp thoại đăng nhập/đăng xuất

8 Mã nguồn minh họa

```
class Mediator(ABC):  
    @abstractmethod  
    def notify(self, sender, event): pass
```

```
@dataclass  
class Button:  
    mediator: Mediator  
  
    def click(self):  
        self.mediator.notify(self, "click")
```

```
@dataclass
class TextBox:
    mediator: Mediator

    text: str = ""
    def get_text(self): return self.text
    def set_text(self, text): self.text = text
```

```
class ConcreteMediator(Mediator):
    def __init__(self, btn: Button, txtbox: TextBox):
        self.button = btn
        self.textbox = txtbox

    def notify(self, sender, event):
        if (sender == self.button)
        and (event == "click"):
            if self.validate_data(
                self.textbox.get_text()
            ): self.save_data()

    def validate_data(self, data): return bool(data)
    def save_data(self): print("Data saved!")
```

9 Ứng dụng thực tế

- **Thư viện giao diện:** WinForms, Qt
- **Hệ thống nhắn tin:** Nhóm chat tập trung
- **Kiến trúc MVC:** Controller là Mediator
- **Lập trình game:** lớp Manager
- **Hệ thống phân tán:** Message broker

10 Trường hợp áp dụng

Sử dụng Mediator khi:

- Khó sửa các lớp vì phụ thuộc chặt chẽ
- Không thể tái sử dụng Component vì phụ thuộc các lớp khác
- Phải tạo quá nhiều lớp kế thừa
- Logic giao tiếp giữa các đối tượng phức tạp

11 Ưu nhược điểm

✓ Ưu Điểm

- Đơn nghĩa vụ
- Thêm được sửa không
- Giảm phụ thuộc chặt chẽ
- Dễ tái sử dụng

X Nhược Điểm

- Mediator phải quản lí quá nhiều thứ
- Khó debug, tăng độ phức tạp

12 Cách áp dụng

1. Xác định các lớp liên quan chặt chẽ
2. Khai báo Mediator trừu tượng
3. Định nghĩa Mediator kế thừa
4. Sửa Component để sử dụng Mediator
5. Components gọi Mediator thay vì gọi nhau trực tiếp

13 So sánh với các mẫu khác

Mediator: Giao tiếp hai chiều, giảm phụ thuộc, tập trung logic

Facade: Giao tiếp một chiều, cung cấp interface đơn, subsystem không biết

Observer: Phân tán giao tiếp, dynamic subscriptions, thông báo sự kiện

14 Các mẫu liên quan

Observer: Mediator dùng Observer để giao tiếp

Command: Gói gọi thành command mà Mediator xử lý

State: Mediator chuyển trạng thái của Component

Strategy: Các chiến lược giao tiếp khác nhau

Facade: Mediator như Facade cho các lớp Component

15 Lỗi thường gặp

Lỗi thường gặp:

- ✗ Mediator quá lớn (God Object)
- ✗ Các Component vẫn gọi nhau trực tiếp
- ✗ Mediator quản lí quá nhiều chi tiết

Cách khắc phục:

- ✓ Giữ interface đơn giản
- ✓ Sử dụng enum cho các loại event
- ✓ Chia nhỏ Mediator
- ✓ Hiểu rõ luồng giao tiếp
- ✓ Test các Component độc lập

16 Tóm tắt

Mẫu Mediator

Giảm phụ thuộc bằng cách tập trung giao tiếp giữa các lớp thông qua đối tượng trung gian.

Thích hợp cho: UI phức tạp, logic trong game, hệ thống phân tán

Cần cẩn thận: Mediator không được quá lớn

Cải thiện: Tính bảo trì, tính tái sử dụng, tính dễ kiểm thử

Cảm ơn vì đã mua sự chú ý!

Nhóm 5